Bilingual Speech Synthesis with FastSpeech 2: An English and Telugu Study

Technical Report – English (Svarah) and Telugu (IndicTTS Telugu) Neural TTS Systems

1. Overview

This work builds a non-autoregressive neural text-to-speech (TTS) pipeline based on FastSpeech 2 for two languages: English, trained on the Svarah corpus, and Telugu, trained on the IndicTTS Telugu corpus. The codebase follows the open-source PyTorch FastSpeech 2 reference, and is adapted to each language through its own corpus, preprocessing rules, and output locations.

The objective is a model that learns phoneme timing, fundamental frequency (F0) and loudness contours, and the mel-frequency spectrogram in a single feed-forward pass, with a HiFi-GAN neural vocoder converting spectrograms into waveforms. The deliverables include per-language configuration files, synthesized audio, plots of the prosody predictor outputs, and a set of five evaluation utterances per language assessed for pronunciation and naturalness.

2. System Design and Method

2.1 Base Implementation

The system derives from the widely used xcmyz PyTorch implementation of FastSpeech 2. Its processing chain consists of:

- A transformer **phoneme encoder** that embeds the input phoneme sequence;
- A **prosody (variance) predictor** that estimates phoneme timing, F0, and loudness;
- A **spectrogram decoder** that reconstructs the mel-frequency spectrogram, followed by a post-net refinement stage;
- A **HiFi-GAN vocoder** that renders the waveform from the spectrogram.

The practical design decisions documented by the reference configuration are also retained:

- F0 and loudness are predicted per phoneme rather than per frame;
- F0 and loudness targets are normalized during preprocessing;
- A 6-layer decoder replaces the 4-layer decoder of the original paper, increasing modelling capacity;
- Gradient clipping and a learning-rate warm-up schedule stabilize optimization;
- Speaker embeddings allow multi-speaker training.

2.2 Retained Architecture Settings

Both language folders keep the reference architecture, as the model configuration files show:

Parameter	Value	Parameter	Value
transformer.encoder_layer	4	transformer.encoder_hidden	256

Parameter	Value	Parameter	Value
transformer.decoder_layer	6	transformer.decoder_hidden	256
transformer.encoder_head	2	transformer.decoder_head	2
variance_predictor.filter_size	256	variance_predictor.kernel_size	3
variance_embedding.pitch_quantization	linear	variance_embedding.energy_quantization	linear
vocoder.model	HiFi-GAN	multi_speaker	true

These values agree with the documentation in `English/config/README.md` and `Telugu/config/README.md`, which stress per-phoneme prosody modelling and a deeper decoder.

2.3 Language-Specific Adaptations

The same network is reused for both languages; only the following elements differ: preprocessing and text-cleaning routines, corpus locations, validation-set size and language tag, lexicon and text-normalization rules, and the output folders for audio and prosody plots.

Setting	English	Telugu
language	en	te
text_cleaners	english_cleaners	english_cleaners
val_size	691	525

Although the architecture is identical, Telugu is the harder modelling problem: its script is morphologically rich and syllabic in structure, which makes phoneme timing and prosody more variable than in English.

3. Data, Preprocessing, and Training Setup

3.1 English Corpus

The English setup (`English/config/English/preprocess.yaml`) reads from `/kaggle/temp/work/raw_data/English` and writes to `/kaggle/temp/work/preprocessed_data/English`. Key options: language *en*, 22,050 Hz sampling, 80 mel channels, phoneme-level F0 and loudness with normalization on, and 691 held-out validation utterances. This suits a clean, regular corpus with a sizeable validation partition for model selection. Phoneme alignment relies on Montreal Forced Aligner (MFA) with the lexicon `lexicon/librispeech-lexicon.txt`.

3.2 Telugu Corpus

The Telugu setup (`Telugu/config/Telugu/preprocess.yaml`) uses the same options with paths `/kaggle/temp/work/raw_data/Telugu` and `/kaggle/temp/work/preprocessed_data/Telugu`, language *te*, and a slightly smaller validation set of 525 utterances. The preprocessing logic is unchanged, but the data are expected to be acoustically and phonologically more complex because of vowel–consonant combinations, orthographic variation, and prosodic phrasing.

3.3 Preprocessing Procedure

Both languages follow the standard FastSpeech 2 sequence:

- **1.** Run `prepare_align.py` to ready the data for alignment.
- **2.** Obtain phoneme-level alignments via MFA or precomputed alignment files.
- **3.** Run `preprocess.py` to extract mel spectrograms, F0, loudness, phoneme durations, and text/phoneme sequences.
- **4.** Store the results under the `preprocessed_data/...` directory.

Audio / feature setting	Value
Sampling rate	22,050 Hz
STFT filter length / window length	1024 / 1024
Hop length	256
Mel channels	80
F0 and loudness resolution	Phoneme level, normalized

Per-phoneme feature prediction is preferred because the configuration notes report that it yields more natural prosody than per-frame prediction.

3.4 Optimization Schedule

Both languages share nearly the same `train.yaml`:

Hyperparameter	Value	Hyperparameter	Value
batch_size	16	total_step	40000
grad_clip_thresh	1.0	save_step	10000
weight_decay	0.0	val_step	1000
warm_up_step	4000	anneal_steps	[28000, 34000]
anneal_rate	0.3		

The schedule combines a warm-up phase with learning-rate reductions at later stages, and checkpoints and logs are kept in separate per-language directories.

4. Loss-Curve Analysis

Loss plots are stored at `Results/English/` and `Results/Telugu/` (`training_loss.png` and `validation_loss.png` in each).

4.1 Convergence Behaviour

- A sharp drop in loss over the first several thousand steps;
- Slower, steady improvement during later steps;
- Settling into a low-loss region after warm-up and learning-rate annealing;
- Validation loss tracking training loss with only a modest gap.

This indicates stable learning of the mapping from phonemes to spectrogram and prosodic targets. Warm-up and annealing are well suited here because the model fits several coupled objectives at once: phoneme timing, F0, loudness, and mel reconstruction.

4.2 Instability and Overfitting

No serious overfitting is evident, though some points deserve care:

- Telugu shows somewhat larger swings in validation loss than English, reflecting harder phonology and prosody;
- The training–validation gap is present but not large, so the model generalizes without heavy memorization;
- Small late-stage oscillations are plausible as the model trades off prosody prediction against spectral reconstruction.

Such behaviour is normal for multi-objective TTS, which must balance audio quality, prosody, and alignment accuracy. Longer training would likely call for extra regularization or extended fine-tuning to reduce the remaining instability.

4.3 English versus Telugu

English converges more smoothly, as its task and acoustic patterns are more uniform. Telugu is harder because of greater phonetic diversity, script-specific prosodic patterns, more complex vowels and syllable structure, and a stronger dependence on language-specific timing and intonation. Telugu therefore tends toward slower convergence and more waveform-level instability on difficult sentences.

5. Evaluation of Synthesized Speech

Generated audio for five utterances per language is stored in `Results/English/test_generated_audios/` and `Results/Telugu/test_generated_audios/`.

5.1 English Utterances

#	Utterance	Assessment
1	Hello, how are you today?	Clear, natural pronunciation and good pacing; final intonation slightly flat but smooth and intelligible.
2	The researchers will conduct an experiment.	Fully intelligible; timing follows phrase structure. Minor roughness on "experiment", where stress and timing are sometimes compressed.
3	Students learn new things every day.	Highly intelligible with stable prosodic flow and even phrase separation; among the most natural English samples.
4	And they went off into the space when earth was burning up or being destroyed	Hardest English item. Long, complex phrasing causes prosodic compression, some rushed words and unstable intonation; meaning is kept but naturalness drops.
5	Karnataka's expansive list of handmade artisanal products are impressive	Mostly understandable but with hesitation-like rhythm and unnatural emphasis; the long, lexically dense sentence reduces smoothness and shows limits on complex structure.

5.2 Telugu Utterances

Note: the Telugu sentences could not be recovered as readable script from the source document, so they are labelled by number here; insert the original Telugu text where indicated.

#	Utterance	Assessment
1	[Telugu greeting phrase – insert original]	Clear, understandable greeting with good naturalness in opening and closing tones; pitch variation is strong but acceptable.
2	[Telugu sentence 2 – insert original]	Very intelligible and natural; short sentence with stable timing and strong clarity. One of the best Telugu results.
3	[Telugu sentence 3 – insert original]	Good overall quality with mostly natural phrase rhythm; some emphasis slightly exaggerated but still fluent.
4	[Telugu sentence 4 – insert original]	Generally understandable and expressive; timing matches clause structure. Slightly robotic phrasing at sentence edges, content remains clear.
5	[Telugu sentence 5 – insert original]	Natural and smooth with good intelligibility and phrase flow; more stable and easier to follow than the long English sentence.

5.3 Overall Quality

In both languages the most natural speech occurs on short, syntactically simple sentences, and quality falls most on long, complex, or heavily inflected ones. Both languages are sensitive to sentence length, but Telugu depends more on phrase-level stability because of its prosodic structure and orthographic complexity.

6. Prosody-Predictor Plots: Timing, F0, and Loudness

Plots for every evaluation utterance are saved under `Results/<Language>/variance_adaptor_plots/` in the subfolders `duration/`, `pitch/`, and `energy/`. They show how the model assigns timing, F0, and loudness to phonemes or syllables.

6.1 Phoneme Timing (Duration)

In both languages, content words and stressed syllables receive longer durations, function words and short connectors get shorter ones, and sentence-final words are usually lengthened. In English utterance 1, timing rises around "hello", "today", and the closing question phrase; in utterance 2, "conduct an experiment" is clearly longer than neighbouring function words, matching natural emphasis. Telugu timing is more variable because prosody there depends strongly on word shape and accent placement: the greeting (utterance 1) and utterance 4 show longer durations on content-heavy words while function-like segments stay short.

6.2 Fundamental Frequency (Pitch)

F0 is modelled per phoneme and normalized. Typical patterns are higher F0 at word starts or emphasized syllables, a gradual fall across the phrase unless a new idea or a question calls for a rise, and a more marked contour near the end of questions. English utterance 1 rises in the first phrase and falls toward the end, as expected for a conversational question, whereas utterance 2 is flatter, a neutral declarative delivery. Telugu plots show stronger local variation: the greeting has a pronounced oscillating contour typical of a question, while utterance 5 progresses more evenly, matching a gentle declarative rhythm.

6.3 Loudness (Energy)

Loudness serves as a proxy for emphasis, with peaks at stressed words, clause boundaries, and key lexical items.

- English utterance 2 peaks around "conduct" and "experiment", a sensible semantic emphasis;
- English utterance 4 has irregular loudness because the long, dense sentence is hard to interpret;
- Telugu utterances 3 and 5 show strong peaks on content words, supporting natural emphasis but occasionally overshooting local intensity.

6.4 Interpretation

The plots confirm that the model learns meaningful prosodic structure: longer timing on content words, F0 movement around phrase boundaries, and loudness concentrated on meaningful words. They also reveal limits: long sentences yield compressed or unstable contours, complex syntax fragments the loudness and F0 shapes, and Telugu prosody stays more variable and less stable than English because of script-specific phonology and lexical complexity.

7. English versus Telugu: Comparison

7.1 English Strengths

English performs better overall: its corpus and preprocessing are cleaner and more regular, sentence-level prosody is more uniform with fewer script-driven phonological ambiguities, and the reference implementation is naturally suited to it. English samples are consequently smoother, more natural, and more consistent in rhythm.

7.2 Telugu Strengths

The Telugu model does well on short, clearly structured sentences. It captures stress and speech rhythm reasonably when the phrase is compact, and short greetings or simple statements can sound natural and clear.

7.3 Limitations and Failure Modes

- Long or syntactically complex sentences give unnatural rhythm;
- Pronunciation can degrade on rare lexical patterns or dense phrasing;
- Some words receive unstable F0 or loudness assignments;
- Telugu is more sensitive to morphological and prosodic complexity;
- Output can sound robotic or patchy when timing and F0 are poorly aligned. These issues are common in small-scale TTS systems.

7.4 Suggested Improvements

- Use a larger, more diverse Telugu dataset;
- Train longer or fine-tune with prosody-specific loss terms;
- Adopt more robust phoneme lexicons and Telugu text normalization;
- Try different vocoder settings and acoustic-model variants;

- Improve tokenization and normalization for language-specific punctuation and script behaviour;
- Evaluate on a larger test set than five representative sentences;
- Add objective metrics (MOS, intelligibility, F0 error) alongside subjective listening.

8. Reproducibility

8.1 Dependencies

Dependencies are listed in `project/requirements.txt`, with per-language files at `project/English/requirements.txt` and `project/Telugu/requirements.txt`. The stack is the typical FastSpeech 2 one: PyTorch plus audio and machine-learning libraries.

8.2 Configuration Files

Under `project/English/config/English/` and `project/Telugu/config/Telugu/`, the files `preprocess.yaml`, `model.yaml`, and `train.yaml` define dataset paths, feature extraction, F0/loudness normalization, the training schedule, and the model architecture.

8.3 Commands

Installation

```
cd project
pip install -r requirements.txt
```

English

```
cd project/English
python prepare_align.py config/English/preprocess.yaml
python preprocess.py config/English/preprocess.yaml
python train.py -p config/English/preprocess.yaml -m config/English/model.
yaml -t config/English/train.yaml
python synthesize.py --text "Hello, how are you today?" --restore_step 400
00 --mode single
  -p config/English/preprocess.yaml -m config/English/model.yaml -t config
/English/train.yaml
```

Telugu

```
cd project/Telugu
python prepare_align.py config/Telugu/preprocess.yaml
python preprocess.py config/Telugu/preprocess.yaml
python train.py -p config/Telugu/preprocess.yaml -m config/Telugu/model.ya
ml -t config/Telugu/train.yaml
python synthesize.py --text "" --restore_step 40000 --mode single
  -p config/Telugu/preprocess.yaml -m config/Telugu/model.yaml -t config/T
elugu/train.yaml
```

These steps match the project README and the per-language configuration layout.

9. Conclusion

The project shows that a FastSpeech 2 system can be built for both English and Telugu from one shared architecture with language-specific preprocessing and configuration. English output is more consistent and natural, while Telugu shows good promise but is

more sensitive to prosodic complexity and longer sentences.

The prosody-predictor plots show that timing, F0, and loudness are learned meaningfully, yet the quality gap between short and long sentences indicates that more data, longer training, and better language-specific normalization would improve both intelligibility and naturalness. The repository is well organized and its configuration files expose the training and inference parameters clearly, so with more diverse data and a longer schedule both models could be substantially improved.